

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

1	Bemutakozás - Kurzus információk - Infrastruktúra szabályzatok - Mentorálás	- Algoritmus vezérlése - A Python nyelv utasításai
2	Python alapok - Bevezető - Pénzváltás (1. verzió) - Pénzváltás (2. verzió)	
3	Alapfogalmak - Alapfogalmak - Alapvető programelemek fogalmak	
4	Python feltételes ciklusok - Röppálya számítás - Röppálya szimuláció	
5	Szoftverfejlesztési életciklus modellek A programozás fázisai	
6	Python sztringek - Python sztringek - Basename meghatározása	
7	Vezérlési szerkezetek - Bevezetés - Szekvenciális vezérlés - Függvények	
8	Python listák - Python list és tuple adattípusok - Python listaszerű adattípusok műveletei - Basename és eltelt idő	
9	Vezérlési szerkezetek Szelekciós vezérlések	
10	Python iterációk Csúszóátlag számítása	
11	Vezérlési szerkezetek Ismétléses vezérlések	
12	Python halmazok, comprehension - Python set és frozenset adattípusok - Tér legtovábbi pontjai	
13	Vezérlési szerkezetek Ismétléses vezérlések	
14	Python dictionary, függvény mint típus - Python dict adattípus - Python comprehension - Python function adattípus	
15	Vezérlési szerkezetek Eljárásvezérlés	
16	Python fájlkezelés Fájlkezelés	
17	Vezérlési szerkezetek Eljárásvezérlés	
18	Python formátált IO Python formátált IO	
19	Adatkezelés, adattárolás - Az adatkezelés szintjei - Az adattárolás jellemzői	
20	Python típusok Python numerikus típusok	
21	Absztrakt adattípusok - Egyszerű adattípusok - Összetett adattípusok	
22	Python típusok	
		- További Python típusok - Python function adattípus
23	Absztrakt adattípusok Összetett adattípusok	
24	Tesztelés - Tesztelés - Tesztelés Python-ban - A pytest	
25	Vizualizáció és egyéb modulok - Fájlkezelő modulok - Vizualizáció	
26	More Python - A Python program végrehajtásának folyamata - Függvényparaméterek és argumentumok - Python unpacking - Kivételkezelés	

Tartalomjegyzék

- 
- Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - Python iterációk**
 - Csúszóátlag számítása
 - Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - Vezérlési szerkezetek**
 - Eljárásvezérlés
 - Python fájlkezelés**
 - Fájlkezelés
 - Vezérlési szerkezetek**
 - Eljárásvezérlés
 - Python formátált IO**
 - Python formátált IO
 - Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - Python típusok**
 - Python numerikus típusok
 - Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - Python típusok**
 - További Python típusok**
 - Python function adattípus
 - Absztrakt adattípusok**
 - Összetett adattípusok
 - Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

- A sztring egy olyan adattípus, melynek értékkészlete (különböző hosszúságú) szövegeket tartalmaz.
- Egy sztring érték felfogható karakterek (betűk, számjegyek, írásjelek, vezérlő karakterek, ...) valamilyen hosszúságú sorozatának.
 - Python-ban nincs külön karakter adattípus, de ebbe most ne gondoljunk bele; az egy (karakter) hosszúságú sztringeket tekintjük karaktereknek.
- Létezik az *üres sztring*, amely nem tartalmaz egyetlen karaktert sem.
- Sztringértéket megadni elsődlegesen sztring literálok segítségével lehet.
- A sztring típusnak rengeteg művelete, metódusa (a típus értékein értelmezett függvénye) van, amelyekkel egy sztring értékéből új értékeket hozhatunk létre.

Sztring literálok

- A sztringek (szövegek, `str`) Python-ban egyfajta listák, karakterek listái
 - *Ez így nem teljesen igaz, pythonban nincs karakter típusú érték, csak 1 hosszúságú sztring!*
- Sztring literálokat (értékeket) többféleképpen lehet megadni
 - Aposztrófok (`'` – *quote*) között
 - Idézőjelek (`"` – *double quote*) között
 - Többsoros sztringeket tripla határolók (`'''` vagy `"""` – „*triple quote*”) között

```
>>> 'Ez_egy_aposztrófok_között_megadott_sztring.'  
'Ez_egy_aposztrófok_között_megadott_sztring.'  
>>> "Ez_egy_idézőjelek_között_megadott_sztring."  
'Ez_egy_idézőjelek_között_megadott_sztring.'  
>>> """Ez_pedig,  
..._mint_az_látszik,  
..._egy_többsorosként_megadott_sztring."""  
'Ez_pedig,\n mint_az_látszik,\negy_többsorosként_megadott_sztring.'
```

Sztring

Speciális karakterek

- A Python sztringekben speciális karaktert *escape szekvenciával* lehet megadni.
 - `\c` – Speciális jelentésű karakter, pl. `\\`, `\'`, `\"`, `\n`, `\t`, ...
 - `\xnn` – Az `nn` hexadecimális kódú Unicode karakter, pl. `'\x40' == '@'`
 - `\unnnn` – Az `nnnn` hexadecimális Unicode kódú karakter, pl. `'\u0150' == 'Ő'`
- A Python két függvénye képes karakterkód és karakter (egyelemű sztring) közötti konverzióra:
 - `ord(c)` – visszaadja a `c` karakter (egyelemű sztring) kódját
 - `chr(i)` – visszaadja az `i` kódú Unicode karaktert (tartalmazó sztringet)

```
>>> print('Igen?\rNem!\n\tIndentálva\b\bva\x21\u2713')
Nem!?

        Indentálva!✓
>>> ord('A')
65
>>> chr(10003)
'✓'
```

Sztring

Speciális sztring literálok

- Unicode sztring literál: `u` prefix (vagy nincs prefix)
 - A Python3-ban ez az alapértelmezett sztring literál típus.
- Nyers (*raw*) sztring literál: `r` prefix
 - Nem történik escape szekvencia helyettesítés.
- Formátum sztring literál (*f-string*): `f` prefix
 - A sztringekben lévő `{` és `}` karakterek közötti rész kifejezésként kiértékelődik, és a megadott formátumspecifikációnak (`:` vagy `!` mögötti rész) megfelelően sztringként behelyettesítődik.

```
>>> print('Van\tescape\tszekvencia')
Van      escape      szekvencia
>>> print(r'Nincs\tescape\tszekvencia')
Nincs\tescape\tszekvencia
>>> v = 1 / 3
>>> f'Egyharmad_{v};_Egy_{3*v:.3f}'
'Egyharmad_{v};_Egy_{3*v:.3f}'
>>> f'Egyharmad_{v};_Egy_{3*v:.3f}'
'Egyharmad_{v};_Egy_{3*v:.3f}'
```

Sztring

Formátumsztringek

- Python-ban több lehetőségünk van arra, hogy egy sztringbe (akár utólag) értékeket szúrjunk be (mivel az `str` *immutable*, ezzel új sztringet hozunk létre).
 - A `%` és `%=` műveletek.
 - A `%` karakterrel kezdődő konverziós specifikációk helyére tudunk értékeket beszúrni.
 - A művelet paramétere egy `tuple`, ami megfelelő számú és típusú értéket tartalmaz.
 - A `.format()` metódus és az `f-string`-ek.
 - A sztringben lévő `{}` és `}` karakterek közötti részeket tudjuk lecserélni.
 - Az `f-string`-ek az aktuális scope változóit használják.
 - Az `.format()` metódusban sorrendben vagy név szerint tudjuk megadni a sztringben helyettesítendő értékeket vagy változók értékeit.

```
>>> 'A_d_az_s.' % (0, 'nulla')
'A_0_az_nulla.'
>>> 'A_{}_az_{}.'.format('egy', 1)
'A_egy_az_1.'
>>> x = 42
```

```
>>> y = 'semmi'
>>> f'A_{x}_az_nem_{y}.'
'A_42_az_nem_semmi.'
>>> '{x}{y}'.format(x=20, y='24')
'2024'
```

Az str (sztring) adattípus műveletei

Operátorok

- egyoperandusú műveletek
 - `len()` karakterek száma
- kétoperandusú műveletek
 - `%` formázás/behelyettesítés
 - `+` összefűzés
 - `*` többszörözés
 - `[]` indexelés
 - `in` tartalmazás
 - `not in` nem tartalmazás
- többoperandusú műveletek
 - `[::]` szeletelés

```
len('AA-23')  
  
"A_válasz:_%d." % (42)  
"A_válasz:_" + str(42)  
"ba" * 2  
"1138"[2]  
'aba' in "kabalababa"  
"kutya" not in 'Farkasfa'  
'ttotídrof'[::-1]
```

Az str (sztring) adattípus műveletei

Relációs műveletek

- kétoperandusú műveletek

`==` egyenlő

`!=` nem egyenlő

`<` lexikografikusan kisebb

`>` lexikografikusan nagyobb

`<=` lexikografikusan kisebb
vagy egyenlő

`>=` lexikografikusan nagyobb
vagy egyenlő

```
'kettő' == 'hat'
'kettő' != '2'
'alma' < 'alma'
'-42' > '+42'
'hat' <= 'hatvan'
'alma' >= 'Alma'
```

Az str (sztring) adattípus műveletei

Gyakoribb/hasznos műveletek (betűmanipuláció)

- Betűmanipuláció

str az eredmény

.lower() kisbetűsítés

.upper() nagybetűsítés

.swapcase() kis/nagybetű csere

.capitalize() első kezdőbetű
nagybetű, a többi kicsi

.title() minden kezdőbetű
nagybetű

```
s.lower()  
s.upper()  
s.swapcase()  
s.capitalize()  
s.title()
```

Az str (sztring) adattípus műveletei

Gyakoribb/hasznos műveletek (törlés/bővítés)

- Sztringmanipuláció

str az eredmény

`.lstrip()` kezdő szóközök eldobása

`.rstrip()` végső szóközök eldobása

`.strip()` kezdő/végső whitespace eldobása

`.center()` középre igazítás

`.ljust()` balra igazítás

`.rjust()` jobbra igazítás

```
s.lstrip()
s.rstrip("\t ")
s.strip()
s.strip('0123456789')
s.center(42)
s.ljust(42)
s.rjust(42)
```

Az str (sztring) adattípus műveletei

Gyakoribb/hasznos műveletek (átalakítás)

- Sztringmanipuláció

str az eredmény

`.replace()` csere

`+=` hozzáfűzés

`*=` meg többszörözés

`.format()` formátumsztring feltöltése

`%=` behelyettesítés

```
s.replace("alma", "körte")
s += ", barack"
s *= 2
"A_válasz: {ans}".format(ans=42)
"A_kérdés: Mennyi {}*{}?".format(6, 7)
a = "A_s:d"
a %= ('válasz', 42)
```

Az str (sztring) adattípus műveletei

Gyakoribb/hasznos műveletek (ellenőrzések)

- Tulajdonság-ellenőrzés

bool az eredmény

`.endswith()`
`.startswith()`
`.islower()`
`.isupper()`
`.istitle()`
`.isspace()`
`.isprintable()`
`.isalpha()`
`.isdecimal()`
`.isalnum()`

```
'száz' .endswith('egy')  
''.startswith('')  
s.islower()  
'nagybetu'.isupper()  
"Kutya".istitle()  
'\t\n'.isspace()  
"Sok\nsor".isprintable()  
'2024'.isalpha()  
'2024'.isdecimal()  
'Y2K'.isalnum()
```

Az str (sztring) adattípus műveletei

Gyakoribb/hasznos műveletek (keresés)

- Keresés

int az eredmény

- .count() előfordulások száma
- .find() első előfordulás keresése
- .index() első előfordulás keresése
- .rfind() utolsó előfordulás keresése
- .rindex() utolsó előfordulás keresése

```
'Szeged'.count('e')  
'Farkasfa'.find('kutya')  
'Hódmezővásárhely'.index('hely')  
'kabalababa'.rfind('aba')  
'Szeged'.rindex('Algyő')
```

Konverzió str (*string*) típusra

- `str(val)`:
 - Bármely típusú értéket `str` típusúvá konvertál



```
>>> str([True, False, None])
'[True, False, None]'
>>> str(42)
'42'
>>> str((12e2, 9j+1))
'(1200.0, (1+9j))'
>>> str([(1,2), {4,3}])
'[(1, 2), 3, 4]'
>>> str({"hat":6, 5:"öt"})
"'hat': 6, 5: 'öt'"
>>> str(print)
'<built-in function print>'
>>> str(type(3))
"<class 'int'>"
```

Az str (sztring) adattípus műveleteinek tulajdonságai

- Az `str` *immutable* típus. Az egyes műveletek nem annak a sztringnek az értékét változtatják meg, amin dolgoznak, hanem egy új sztringet adnak vissza!

```
>>> a = b = "Alapérték"
>>> a is b
True
>>> b += "Bővítés"
>>> a is b
False
>>> a
'Alapérték'
>>> b
'AlapértékBővítés'
```

- Ha egy sztringet mint karakterek sorozatát értelmezzük, akkor beszélhetünk a sztring hosszáról és adott pozícióban lévő karaktereiről.
- Az `s` sztring hosszát (karaktereinek számát) a `len(s)` függvényhívással kaphatjuk meg.
- Az `s` sztring n -dik karakterét az `s[n]` kifejezéssel kaphatjuk meg.
 - A sztringek indexelése 0-tól kezdődik („hanyadik karakter?” helyett „hány karakter előzi meg?”).
 - Az utolsó érvényes index a sztring hossza mínusz 1.
 - Negatív számmal is lehet indexelni, ekkor a sztring végétől előre számolunk („hátról a hányadik karakter?”). A -1 index a sztring utolsó karakterét jelöli ki.
 - Sztring esetén az indexelés művelet egy egy karakter hosszúságú sztringet fog visszaadni (Python-ban ugyanis nincs karakter típus).

Szeletek (slice)

str típus esetén

- Egy adott sztring (index szerinti) részsstringjét Python-ban szeletek segítségével kapjuk meg.
- Az `s` sztring n -dik karakterétől m -dik karakteréig (m -et **nem** beleértve) minden k -adik karakterből képzett sztringet az `s[n:m:k]` kifejezéssel kaphatjuk meg.
- Negatív k érték esetén n -től a sztring eleje felé lépkedünk.
- Ha a kezdő és végpont nem a haladási iránynak megfelelően van megadva, üres sztringet kapunk.
- k (és a második kettőspont is) elhagyható, ilyenkor az alapértelmezett értéke 1.
- n elhagyható, ilyenkor az alapértelmezett értéke nem korlátozza az eredmény szelet értékét.
- m elhagyható, ilyenkor az alapértelmezett értéke nem korlátozza az eredmény szelet értékét.

Szeletek (slice)

str típus esetén

- Példák

```
>>> s = "Abrakadabra"
>>> s[:]
'Abrakadabra'
>>> s[4]      # Ez index, nem szelet!
'k'
>>> s[-3]     # Ez is index!
'b'

>>> s[2:5]
'rak'
```

```
>>> s[2:9:3]
'rab'
>>> s[: -6:]
'Abrak'
>>> s[-3:1:-3]
'bar'
>>> s[0:len(s):-1]
''

>>> 'Az_nem_menza'[::-1]
'aznem men zA'
```

Sztringek bejárása

Elemek (karakterek) szerint

- A ciklussal sorban megkapjuk a sztring elemeit (karaktereit), de azt nem fogjuk tudni, hogy az adott karakternek mi volt az indexe (hacsak külön nem számoljuk).

```
>>> s = 'Macska'
>>> for c in s:
...     print(c)
...
'M'
'a'
'c'
's'
'k'
'a'
```

Sztringek bejárása

Indexek szerint

- A ciklussal sorban megkapjuk a sztring elemeinek (karaktereinek) az indexeit. Az index segítségével esetleg magához a karakterhez is hozzáférhetünk.

```
>>> s = 'Macska'
>>> for i in range(len(s)):
...     print(i, s[i], s[-i])
...
0 'M' 'M'
1 'a' 'a'
2 'c' 'k'
3 's' 's'
4 'k' 'c'
5 'a' 'a'
```

Sztringek bejárása

Index-elem párok szerint

- A ciklussal sorban megkapjuk a sztring indexeit és az adott indexeknek megfelelő elemeit (karaktereit) is.

```
>>> s = 'Macska'
>>> for i, c in enumerate(s):
...     print(i, c)
...
0 'M'
1 'a'
2 'c'
3 's'
4 'k'
5 'a'
```

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása**
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A basename program

- A linux-os basename program egy útvonalból (*path*) megadja az adott fájl nevét, a könyvtárak nélkül.
- A program algoritmus nem túl bonyolult. Az útvonalleírás egy sztring, amelyben az utolsó útvonalelválasztó `'/'` karakter mögötti részt kell visszaadni. Ha az utolsó karakter a `'/'`, akkor az útvonal egy könyvtárat reprezentál, a fájlnev „üres”. Ha nincs `'/'` a leírásban, akkor az egész egyetlen fájlnev.
- Vagyis meg kell határozni az utolsó `'/'` karakter indexét, és a sztring e mögötti szeletét kell visszaadni.
- Ha ügyesen csináljuk, a speciális eseteket nem kell külön kezelnünk.

A basename program

Elem szerinti bejárással

- Az első megoldásunkban a `lastsep` változó jelöli a (sorban utoljára) megtalált `'/'` karakter pozícióját (pontosabban az utána következő pozíciót, ahonnan a szelet indul).
- Mivel itt a karaktereket iteráljuk, az indexet/pozíciót magunknak kell számolnunk, erre használjuk az `i` változót.
- Ha nincs `'/'` karakter a sztringben, akkor a szelet a sztring elejéről, a 0-ás indextől kell, hogy induljon.



Basename (elem bejárással)

basename_iterate.py [1–25]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2025-09-16
6cc-by-nc-nd
7
8Adott egy (linuxos) útvonalleírás, nyerjük ki belőle
9a fájl nevét (az útvonal nélkül).
10"""
11def basename(path):
12    i, lastsep = 0, 0
13    for c in path:
14        i += 1
15        if c == '/':
16            lastsep = i
17    return path[lastsep:]
18
19def main():
20    path = input()
21    base = basename(path)
22    print(base)
23
24if __name__ == "__main__":
25    main()
```

A basename program

Index szerinti bejárással

- Az következő megoldásunkban a `lastsep` változó jelöli a (sorban utoljára) megtalált `'/'` karakter pozícióját.
- A megfelelő szeletet a `lastsep + 1` pozíciótól indulva kapjuk.
- A `lastsep` értéke kezdetben `-1`, ami jól fog működni, ha nincs egyetlen `'/'` karakter sem a sztringben.



Basename (index bejárással)

basename_index.py [1–24]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2025-09-16
6cc-by-nc-nd
7
8Adott egy (linuxos) útvonalleírás, nyerjük ki belőle
9a fájl nevét (az útvonal nélkül).
10"""
11def basename(path):
12    lastsep = -1
13    for i in range(len(path)):
14        if path[i] == '/':
15            lastsep = i
16    return path[lastsep+1:]
17
18def main():
19    path = input()
20    base = basename(path)
21    print(base)
22
23if __name__ == "__main__":
24    main()
```

A basename program

Index-elem párok szerinti bejárással

- Az előző logika, csak a megvalósítás változik.



Basename (index-elem páros bejárással)

basename_enumerate.py [1–24]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2025-09-16
6cc-by-nc-nd
7
8Adott egy (linuxos) útvonalleírás, nyerjük ki belőle
9a fájl nevét (az útvonal nélkül).
10"""
11def basename(path):
12    lastsep = -1
13    for i, c in enumerate(path):
14        if c == '/':
15            lastsep = i
16    return path[lastsep+1:]
17
18def main():
19    path = input()
20    base = basename(path)
21    print(base)
22
23if __name__ == "__main__":
24    main()
```

A basename program

A sztring típus egyik módszerét használva

- A Python `str` típusának rengeteg művelete van. Van-e ezek között olyan, amit fel tudnánk használni a feladat megoldásához?
- Van! Az `rfind()` módszer pontosan azt csinálja, ami nekünk kell: megkeresi egy karakter (rész-sztring) utolsó előfordulási helyét (és `-1`-et ad vissza, ha nem találja a keresett rész-sztringet).



Basename (Python módon)

basename_find.py [1–21]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2025-09-16
6cc-by-nc-nd
7
8Adott egy (linuxos) útvonalleírás, nyerjük ki belőle
9a fájl nevét (az útvonal nélkül).
10"""
11def basename(path):
12    lastsep = path.rfind('/')
13    return path[lastsep+1:]
14
15def main():
16    path = input()
17    base = basename(path)
18    print(base)
19
20if __name__ == "__main__":
21    main()
```

A basename program

Parancssori argumentumból

- El lehetne érni azt, hogy a programunk ne bekérje az útvonalat, hanem parancssorban kapja meg, mint az eredeti basename program?
- Igen! a `sys` modul `argv` változója pont erre való. Hogy a `[1]` miért kell oda, arról majd a listáknál beszélgetünk.



Basename (parancssori paraméterből)

basename_args.py [1–21]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2025-09-16
6cc-by-nc-nd
7
8Adott egy (linuxos) útvonalleírás, nyerjük ki belőle
9a fájlnevét (az útvonal nélkül).
10"""
11from sys import argv
12
13def basename(path):
14    lastsep = path.rfind('/')
15    return path[lastsep+1:]
16
17def main():
18    print(basename(argv[1]))
19
20if __name__ == "__main__":
21    main()
```